Synopsis:
This project is a competitive minigames-cabinet in which 2 players complete successively difficult games, and the first player to lose 3 loses the overall competition.

Description:
The application is structured around a single event driven main loop across Python and PyGame. This loop first gathers input, then performs state updates, and renders the game state. Broadly, the Game object is responsible for screens and navigation, Session for match rules and parameters, Minigame for actual gameplay, and TCPServer for networking.

The host sends the minigame type, seed, difficulty, and round start time to clients. From this, clients generate identical minigame content and run them locally. When a client completes the minigame, it sends the game result to the session and host, where the timestamp and elapsed time are used to compute the winner. Clients are assumed to not cheat for this project. The Game object operates the main event loop and the current screen. Navigation states and PyGame input are routed to listeners, specifically the transitions from menu to instructions to countdown to play to results to the end screen. It is also responsible for creating a Session object depending on the host's mode selection.

Game matches are controlled by a session. The Session object routes initial minigame information to clients, primarily the difficulty and seed for the incumbent game. Upon round end, Session uses the results of the games to determine the winner, and deducts player lives. Session then increases the difficulty and selects the next minigame. The Player object stores player identity and life count, listening to Session for game outcome. Game outcome is normalized as a GameResult object outside the session. Session's decision does not involve minigame rules or other minigame information.

Multiplayer is peer hosted, where a client creates a session, displays its ID which another player joins. TCPServer transports messages over TCP. It is responsible for discarding incorrect messages, interpreting types of messages, forwarding contents of such messages to the session, and reporting whether the clients are still connected. Messages are queued within a background thread for the main loop to interpret without interruption.

Minigame is an abstract class defining the game instructions, game logic execution, and game conclusion parameters. Minigames generate content according to seed and difficulty heuristics, e.g. game length, introducing new mechanics, faster or more complex player input requirements, etc. Minigame types are defined by minigame subclasses. The game supports seven minigames, each of which define end conditions uniquely. Rock-Paper-Scissors depends on the opponent's selection. Reaction, DDR, Pizza Cutting, and Simon compare the players' scores. Minesweeper depends on either survival or completion time comparison, and Maze compares completion time.

In the multiplayer mode, clients synchronize minigame start parameters and results, but game state is computed locally and asynchronously. The host's Session object is authoritative, and produces the minigame type, seed, difficulty, and time limit. This Session is responsible for starting the round and broadcasting outcome. Clients cannot influence the other's state without interacting with this Session. It is possible to serve opponent game states continuously via Session to each client, however this is not a requirement among the existing game pool. Singleplayer mode creates a Session without a TCPServer, but otherwise uses the same basic architecture, where a Minigame produces a GameResult which the Session interprets and the game ends when the player loses their lives. Rock Paper Scissors simulates an opponent in singleplayer, however.